CS224N Assignment 1: Exploring Word Vectors (25 Points)

Due 4:30pm, Tue January 13th 2026

Welcome to CS224N!

Before you start, make sure you **read the README.md** in the same directory as this notebook for important setup information. You need to install some Python libraries before you can successfully do this assignment. A lot of code is provided in this notebook, and we highly encourage you to read and understand it as part of the learning :)

If you aren't super familiar with Python, Numpy, or Matplotlib, we recommend you check out the review session on Friday. The session will be recorded and the material will be made available on our website. The CS231N Python/Numpy tutorial is also a great resource.

Assignment Notes: Please make sure to save the notebook as you go along. Submission Instructions are located at the bottom of the notebook.

In [1]:
```
# All Import Statements Defined Here
# Note: Do not add to this list.
# ----------------

import sys
assert sys.version_info[0] == 3
assert sys.version_info[1] >= 8

from platform import python_version
assert int(python_version().split(".")[1]) >= 5, "Please upgrade your Python versio
    the README.md file found in the same directory as this notebook. Your Python ve

from gensim.models import KeyedVectors
from gensim.test.utils import datapath
import pprint
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [10, 5]

from datasets import load_dataset
imdb_dataset = load_dataset("stanfordnlp/imdb", name="plain_text")

import re
import numpy as np
import random
import scipy as sp
from sklearn.decomposition import TruncatedSVD
from sklearn.decomposition import PCA

START_TOKEN = '<START>'
END_TOKEN = '<END>'
```

```
NUM_SAMPLES = 150

np.random.seed(0)
random.seed(0)
# ----------------
```

Word Vectors

Word Vectors are often used as a fundamental component for downstream NLP tasks, e.g. question answering, text generation, translation, etc., so it is important to build some intuitions as to their strengths and weaknesses. Here, you will explore two types of word vectors: those derived from *co-occurrence matrices*, and those derived via *GloVe*.

Note on Terminology: The terms "word vectors" and "word embeddings" are often used interchangeably. The term "embedding" refers to the fact that we are encoding aspects of a word's meaning in a lower dimensional space. As Wikipedia states, "*conceptually it involves a mathematical embedding from a space with one dimension per word to a continuous vector space with a much lower dimension*".

Part 1: Count-Based Word Vectors (10 points)

Most word vector models start from the following idea:

You shall know a word by the company it keeps (Firth, J. R. 1957:11)

Many word vector implementations are driven by the idea that similar words, i.e., (near) synonyms, will be used in similar contexts. As a result, similar words will often be spoken or written along with a shared subset of words, i.e., contexts. By examining these contexts, we can try to develop embeddings for our words. With this intuition in mind, many "old school" approaches to constructing word vectors relied on word counts. Here we elaborate upon one of those strategies, *co-occurrence matrices* (for more information, see here or here).

Co-Occurrence

A co-occurrence matrix counts how often things co-occur in some environment. Given some word w_i occurring in the document, we consider the *context window* surrounding w_i. Supposing our fixed window size is n, then this is the n preceding and n subsequent words in that document, i.e. words $w_{i-n} \dots w_{i-1}$ and $w_{i+1} \dots w_{i+n}$. We build a *co-occurrence matrix* M, which is a symmetric word-by-word matrix in which M_{ij} is the number of times w_j appears inside w_i's window among all documents.

Example: Co-Occurrence with Fixed Window of n=1:

Document 1: "all that glitters is not gold"

Document 2: "all is well that ends well"

*	<START>	all	that	glitters	is	not	gold	well	ends	<END>
<START>	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
<END>	0	0	0	0	0	0	1	1	0	0

In NLP, we commonly use `<START>` and `<END>` tokens to mark the beginning and end of sentences, paragraphs, or documents. These tokens are included in co-occurrence counts, encapsulating each document, for example: "`<START>` All that glitters is not gold `<END>`".

The matrix rows (or columns) provide word vectors based on word-word co-occurrence, but they can be large. To reduce dimensionality, we employ Singular Value Decomposition (SVD), akin to PCA, selecting the top k principal components. The SVD process decomposes the co-occurrence matrix A into singular values in the diagonal S matrix and new, shorter word vectors in U_k.

This dimensionality reduction maintains semantic relationships; for instance, *doctor* and *hospital* will be closer than *doctor* and *dog*.

For those unfamiliar with eigenvalues and SVD, a beginner-friendly introduction to SVD is available here. Additional resources for in-depth understanding include lectures 7, 8, and 9 of CS168, providing high-level treatment of these algorithms. For practical implementation, utilizing pre-programmed functions from Python packages like numpy, scipy, or sklearn is recommended. While applying full SVD to large corpora can be memory-intensive, scalable techniques such as Truncated SVD exist for extracting the top k vector components efficiently.

Plotting Co-Occurrence Word Embeddings

Here, we will be using the Large Movie Review Dataset. This is a dataset for binary sentiment classification containing substantially more data than previous benchmark datasets. We provide a set of 25,000 highly polar movie reviews for training, and 25,000 for testing. There

is additional unlabeled data for use as well. We provide a `read_corpus` function below that pulls out the text of a movie review from the dataset. The function also adds `<START>` and `<END>` tokens to each of the documents, and lowercases words. You do **not** have to perform any other kind of pre-processing.

In [2]:
```
def read_corpus():
    """ Read files from the Large Movie Review Dataset.
        Params:
            category (string): category name
        Return:
            list of lists, with words from each of the processed files
    """
    files = imdb_dataset["train"]["text"][:NUM_SAMPLES]
    return [[START_TOKEN] + [re.sub(r'[^\w]', '', w.lower()) for w in f.split(" ")]
```

Let's have a look what these documents are like....

In [3]:
```
imdb_corpus = read_corpus()
pprint.pprint(imdb_corpus[:3], compact=True, width=100)
print("corpus size: ", len(imdb_corpus[0]))
```

```
[['<START>', 'i', 'rented', 'i', 'am', 'curiousyellow', 'from', 'my', 'video', 'stor
e', 'because',
  'of', 'all', 'the', 'controversy', 'that', 'surrounded', 'it', 'when', 'it', 'wa
s', 'first',
  'released', 'in', '1967', 'i', 'also', 'heard', 'that', 'at', 'first', 'it', 'wa
s', 'seized',
  'by', 'us', 'customs', 'if', 'it', 'ever', 'tried', 'to', 'enter', 'this', 'countr
y', 'therefore',
  'being', 'a', 'fan', 'of', 'films', 'considered', 'controversial', 'i', 'really',
'had', 'to',
  'see', 'this', 'for', 'myselfbr', 'br', 'the', 'plot', 'is', 'centered', 'around',
'a', 'young',
  'swedish', 'drama', 'student', 'named', 'lena', 'who', 'wants', 'to', 'learn', 'ev
erything',
  'she', 'can', 'about', 'life', 'in', 'particular', 'she', 'wants', 'to', 'focus',
'her',
  'attentions', 'to', 'making', 'some', 'sort', 'of', 'documentary', 'on', 'what',
'the', 'average',
  'swede', 'thought', 'about', 'certain', 'political', 'issues', 'such', 'as', 'th
e', 'vietnam',
  'war', 'and', 'race', 'issues', 'in', 'the', 'united', 'states', 'in', 'between',
'asking',
  'politicians', 'and', 'ordinary', 'denizens', 'of', 'stockholm', 'about', 'their',
'opinions',
  'on', 'politics', 'she', 'has', 'sex', 'with', 'her', 'drama', 'teacher', 'classma
tes', 'and',
  'married', 'menbr', 'br', 'what', 'kills', 'me', 'about', 'i', 'am', 'curiousyello
w', 'is',
  'that', '40', 'years', 'ago', 'this', 'was', 'considered', 'pornographic', 'reall
y', 'the', 'sex',
  'and', 'nudity', 'scenes', 'are', 'few', 'and', 'far', 'between', 'even', 'then',
'its', 'not',
  'shot', 'like', 'some', 'cheaply', 'made', 'porno', 'while', 'my', 'countrymen',
'mind', 'find',
  'it', 'shocking', 'in', 'reality', 'sex', 'and', 'nudity', 'are', 'a', 'major', 's
taple', 'in',
  'swedish', 'cinema', 'even', 'ingmar', 'bergman', 'arguably', 'their', 'answer',
'to', 'good',
  'old', 'boy', 'john', 'ford', 'had', 'sex', 'scenes', 'in', 'his', 'filmsbr', 'b
r', 'i', 'do',
  'commend', 'the', 'filmmakers', 'for', 'the', 'fact', 'that', 'any', 'sex', 'show
n', 'in', 'the',
  'film', 'is', 'shown', 'for', 'artistic', 'purposes', 'rather', 'than', 'just', 't
o', 'shock',
  'people', 'and', 'make', 'money', 'to', 'be', 'shown', 'in', 'pornographic', 'thea
ters', 'in',
  'america', 'i', 'am', 'curiousyellow', 'is', 'a', 'good', 'film', 'for', 'anyone',
'wanting',
  'to', 'study', 'the', 'meat', 'and', 'potatoes', 'no', 'pun', 'intended', 'of', 's
wedish',
  'cinema', 'but', 'really', 'this', 'film', 'doesnt', 'have', 'much', 'of', 'a', 'p
lot', '<END>'],
 ['<START>', 'i', 'am', 'curious', 'yellow', 'is', 'a', 'risible', 'and', 'pretentio
us', 'steaming',
  'pile', 'it', 'doesnt', 'matter', 'what', 'ones', 'political', 'views', 'are', 'be
cause', 'this',
```

```
  'film', 'can', 'hardly', 'be', 'taken', 'seriously', 'on', 'any', 'level', 'as',
'for', 'the',
  'claim', 'that', 'frontal', 'male', 'nudity', 'is', 'an', 'automatic', 'nc17', 'th
at', 'isnt',
  'true', 'ive', 'seen', 'rrated', 'films', 'with', 'male', 'nudity', 'granted', 'th
ey', 'only',
  'offer', 'some', 'fleeting', 'views', 'but', 'where', 'are', 'the', 'rrated', 'fil
ms', 'with',
  'gaping', 'vulvas', 'and', 'flapping', 'labia', 'nowhere', 'because', 'they', 'don
t', 'exist',
  'the', 'same', 'goes', 'for', 'those', 'crappy', 'cable', 'shows', 'schlongs', 'sw
inging', 'in',
  'the', 'breeze', 'but', 'not', 'a', 'clitoris', 'in', 'sight', 'and', 'those', 'pr
etentious',
  'indie', 'movies', 'like', 'the', 'brown', 'bunny', 'in', 'which', 'were', 'treate
d', 'to', 'the',
  'site', 'of', 'vincent', 'gallos', 'throbbing', 'johnson', 'but', 'not', 'a', 'tra
ce', 'of',
  'pink', 'visible', 'on', 'chloe', 'sevigny', 'before', 'crying', 'or', 'implying',
  'doublestandard', 'in', 'matters', 'of', 'nudity', 'the', 'mentally', 'obtuse', 's
hould', 'take',
  'into', 'account', 'one', 'unavoidably', 'obvious', 'anatomical', 'difference', 'b
etween', 'men',
  'and', 'women', 'there', 'are', 'no', 'genitals', 'on', 'display', 'when', 'actres
ses', 'appears',
  'nude', 'and', 'the', 'same', 'cannot', 'be', 'said', 'for', 'a', 'man', 'in', 'fa
ct', 'you',
  'generally', 'wont', 'see', 'female', 'genitals', 'in', 'an', 'american', 'film',
'in',
  'anything', 'short', 'of', 'porn', 'or', 'explicit', 'erotica', 'this', 'alleged',
  'doublestandard', 'is', 'less', 'a', 'double', 'standard', 'than', 'an', 'admitted
ly',
  'depressing', 'ability', 'to', 'come', 'to', 'terms', 'culturally', 'with', 'the',
'insides',
  'of', 'womens', 'bodies', '<END>'],
 ['<START>', 'if', 'only', 'to', 'avoid', 'making', 'this', 'type', 'of', 'film', 'i
n', 'the',
  'future', 'this', 'film', 'is', 'interesting', 'as', 'an', 'experiment', 'but', 't
ells', 'no',
  'cogent', 'storybr', 'br', 'one', 'might', 'feel', 'virtuous', 'for', 'sitting',
'thru', 'it',
  'because', 'it', 'touches', 'on', 'so', 'many', 'important', 'issues', 'but', 'i
t', 'does', 'so',
  'without', 'any', 'discernable', 'motive', 'the', 'viewer', 'comes', 'away', 'wit
h', 'no', 'new',
  'perspectives', 'unless', 'one', 'comes', 'up', 'with', 'one', 'while', 'ones', 'm
ind', 'wanders',
  'as', 'it', 'will', 'invariably', 'do', 'during', 'this', 'pointless', 'filmbr',
'br', 'one',
  'might', 'better', 'spend', 'ones', 'time', 'staring', 'out', 'a', 'window', 'at',
'a', 'tree',
  'growingbr', 'br', '', '<END>']]
corpus size:  290
```

Question 1.1: Implement `distinct_words` [code] (2 points)

Write a method to work out the distinct words (word types) that occur in the corpus.

You can use `for` loops to process the input `corpus` (a list of list of strings), but try using Python list comprehensions (which are generally faster). In particular, this may be useful to flatten a list of lists. If you're not familiar with Python list comprehensions in general, here's more information.

Your returned `corpus_words` should be sorted. You can use python's `sorted` function for this.

You may find it useful to use Python sets to remove duplicate words.

In [8]:
```
def distinct_words(corpus):
    """ Determine a list of distinct words for the corpus.
        Params:
            corpus (list of list of strings): corpus of documents
        Return:
            corpus_words (list of strings): sorted list of distinct words across th
            n_corpus_words (integer): number of distinct words across the corpus
    """
    corpus_words = []
    n_corpus_words = -1

    # ------------------
    # Write your implementation here.
    all_words = [word for words in corpus for word in words]
    corpus_words = sorted(list(set(all_words)))
    n_corpus_words = len(corpus_words)

    # ------------------

    return corpus_words, n_corpus_words
```

In [9]:
```
# ---------------------
# Run this sanity check
# Note that this not an exhaustive check for correctness.
# ---------------------

# Define toy corpus
test_corpus = ["{} All that glitters isn't gold {}".format(START_TOKEN, END_TOKEN).
test_corpus_words, num_corpus_words = distinct_words(test_corpus)

# Correct answers
ans_test_corpus_words = sorted([START_TOKEN, "All", "ends", "that", "gold", "All's"
ans_num_corpus_words = len(ans_test_corpus_words)

# Test correct number of words
assert(num_corpus_words == ans_num_corpus_words), "Incorrect number of distinct wor

# Test correct words
assert (test_corpus_words == ans_test_corpus_words), "Incorrect corpus_words.\nCorr

# Print Success
```

```
print ("-" * 80)
print("Passed All Tests!")
print ("-" * 80)
```

```
--------------------------------------------------------------------------------
Passed All Tests!
--------------------------------------------------------------------------------
```

Question 1.2: Implement `compute_co_occurrence_matrix` [code] (3 points)

Write a method that constructs a co-occurrence matrix for a certain window-size n (with a default of 4), considering words n before and n after the word in the center of the window. Here, we start to use `numpy (np)` to represent vectors, matrices, and tensors. If you're not familiar with NumPy, there's a NumPy tutorial in the second half of this cs231n Python NumPy tutorial.

In [12]:
```
def compute_co_occurrence_matrix(corpus, window_size=4):
    """ Compute co-occurrence matrix for the given corpus and window_size (default

        Note: Each word in a document should be at the center of a window. Words ne
              number of co-occurring words.

              For example, if we take the document "<START> All that glitters is no
              "All" will co-occur with "<START>", "that", "glitters", "is", and "no

        Params:
            corpus (list of list of strings): corpus of documents
            window_size (int): size of context window
        Return:
            M (a symmetric numpy matrix of shape (number of unique words in the cor
                Co-occurence matrix of word counts.
                The ordering of the words in the rows/columns should be the same as
            word2ind (dict): dictionary that maps word to index (i.e. row/column nu
    """
    words, n_words = distinct_words(corpus)
    M = None
    word2ind = {}

    # ------------------
    # Write your implementation here.
    M = np.zeros(shape=(n_words, n_words))
    word2ind = {word: i for i, word in enumerate(words)}
    for words in corpus:
        for i in range(len(words)):
            sweep = min(len(words)-1, i+window_size)
            for j in range(i+1, sweep + 1):
                word1Idx = word2ind[words[i]]
                word2Idx = word2ind[words[j]]
                M[word1Idx][word2Idx] += 1
                M[word2Idx][word1Idx] += 1
    # ------------------
```

```
    return M, word2ind
```

In [13]:

```
# ---------------------
# Run this sanity check
# Note that this is not an exhaustive check for correctness.
# ---------------------

# Define toy corpus and get student's co-occurrence matrix
test_corpus = ["{} All that glitters isn't gold {}".format(START_TOKEN, END_TOKEN).
M_test, word2ind_test = compute_co_occurrence_matrix(test_corpus, window_size=1)

# Correct M and word2ind
M_test_ans = np.array(
    [[0., 0., 0., 0., 0., 0., 1., 0., 0., 1.,],
     [0., 0., 1., 1., 0., 0., 0., 0., 0., 0.,],
     [0., 1., 0., 0., 0., 0., 0., 0., 1., 0.,],
     [0., 1., 0., 0., 0., 0., 0., 0., 0., 1.,],
     [0., 0., 0., 0., 0., 0., 0., 0., 1., 1.,],
     [0., 0., 0., 0., 0., 0., 0., 1., 1., 0.,],
     [1., 0., 0., 0., 0., 0., 0., 1., 0., 0.,],
     [0., 0., 0., 0., 0., 1., 1., 0., 0., 0.,],
     [0., 0., 1., 0., 1., 1., 0., 0., 0., 1.,],
     [1., 0., 0., 1., 1., 0., 0., 0., 1., 0.,]]
)
ans_test_corpus_words = sorted([START_TOKEN, "All", "ends", "that", "gold", "All's"
word2ind_ans = dict(zip(ans_test_corpus_words, range(len(ans_test_corpus_words))))

# Test correct word2ind
assert (word2ind_ans == word2ind_test), "Your word2ind is incorrect:\nCorrect: {}\n

# Test correct M shape
assert (M_test.shape == M_test_ans.shape), "M matrix has incorrect shape.\nCorrect:

# Test correct M values
for w1 in word2ind_ans.keys():
    idx1 = word2ind_ans[w1]
    for w2 in word2ind_ans.keys():
        idx2 = word2ind_ans[w2]
        student = M_test[idx1, idx2]
        correct = M_test_ans[idx1, idx2]
        if student != correct:
            print("Correct M:")
            print(M_test_ans)
            print("Your M: ")
            print(M_test)
            raise AssertionError("Incorrect count at index ({}, {})=({}, {}) in mat

# Print Success
print ("-" * 80)
print("Passed All Tests!")
print ("-" * 80)
```

```
--------------------------------------------------------------------------------
Passed All Tests!
--------------------------------------------------------------------------------
```

Question 1.3: Implement `reduce_to_k_dim` [code] (1 point)

Construct a method that performs dimensionality reduction on the matrix to produce k-dimensional embeddings. Use SVD to take the top k components and produce a new matrix of k-dimensional embeddings.

Note: All of numpy, scipy, and scikit-learn (`sklearn`) provide *some* implementation of SVD, but only scipy and sklearn provide an implementation of Truncated SVD, and only sklearn provides an efficient randomized algorithm for calculating large-scale Truncated SVD. So please use sklearn.decomposition.TruncatedSVD.

In [14]:
```
def reduce_to_k_dim(M, k=2):
    """ Reduce a co-occurence count matrix of dimensionality (num_corpus_words, num
        to a matrix of dimensionality (num_corpus_words, k) using the following SVD
            - http://scikit-learn.org/stable/modules/generated/sklearn.decompositio

        Params:
            M (numpy matrix of shape (number of unique words in the corpus , number
            k (int): embedding size of each word after dimension reduction
        Return:
            M_reduced (numpy matrix of shape (number of corpus words, k)): matrix o
                    In terms of the SVD from math class, this actually returns U *
    """
    n_iters = 10     # Use this parameter in your call to `TruncatedSVD`
    M_reduced = None
    print("Running Truncated SVD over %i words..." % (M.shape[0]))

    # ------------------
    # Write your implementation here.
    svd = TruncatedSVD(n_components=k, n_iter=n_iters, random_state=42)
    M_reduced = svd.fit_transform(M)

    # ------------------

    print("Done.")
    return M_reduced
```

In [15]:
```
# ---------------------
# Run this sanity check
# Note that this is not an exhaustive check for correctness
# In fact we only check that your M_reduced has the right dimensions.
# ---------------------

# Define toy corpus and run student code
test_corpus = ["{} All that glitters isn't gold {}".format(START_TOKEN, END_TOKEN).
M_test, word2ind_test = compute_co_occurrence_matrix(test_corpus, window_size=1)
M_test_reduced = reduce_to_k_dim(M_test, k=2)

# Test proper dimensions
assert (M_test_reduced.shape[0] == 10), "M_reduced has {} rows; should have {}".for
assert (M_test_reduced.shape[1] == 2), "M_reduced has {} columns; should have {}".f
```

```
# Print Success
print ("-" * 80)
print("Passed All Tests!")
print ("-" * 80)
```

```
Running Truncated SVD over 10 words...
Done.
--------------------------------------------------------------------------------
Passed All Tests!
--------------------------------------------------------------------------------
```

Question 1.4: Implement `plot_embeddings` [code] (1 point)

Here you will write a function to plot a set of 2D vectors in 2D space. For graphs, we will use Matplotlib (`plt`).

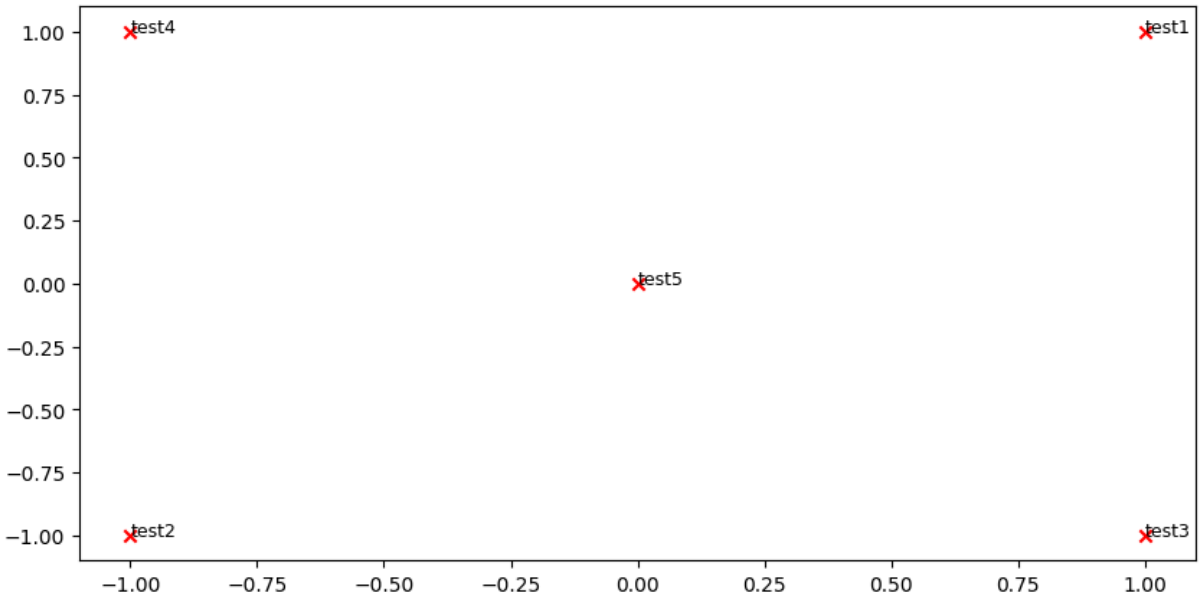
For this example, you may find it useful to adapt this code. In the future, a good way to make a plot is to look at the Matplotlib gallery, find a plot that looks somewhat like what you want, and adapt the code they give.

In [21]:
```
def plot_embeddings(M_reduced, word2ind, words):
    """ Plot in a scatterplot the embeddings of the words specified in the list "wo
        NOTE: do not plot all the words listed in M_reduced / word2ind.
        Include a label next to each point.

        Params:
            M_reduced (numpy matrix of shape (number of unique words in the corpus
            word2ind (dict): dictionary that maps word to indices for matrix M
            words (list of strings): words whose embeddings we want to visualize
    """

    # ------------------
    # Write your implementation here.
    fig, ax = plt.subplots()
    for word in words:
        wordIdx = word2ind[word]
        x, y = M_reduced[wordIdx]
        ax.scatter(x, y, marker='x', color='red')
        ax.text(x, y, word, fontsize=9)

    # ------------------
```

In [22]:
```
# ---------------------
# Run this sanity check
# Note that this is not an exhaustive check for correctness.
# The plot produced should look like the included file question_1.4_test.png
# ---------------------

print ("-" * 80)
print ("Outputted Plot:")

M_reduced_plot_test = np.array([[1, 1], [-1, -1], [1, -1], [-1, 1], [0, 0]])
word2ind_plot_test = {'test1': 0, 'test2': 1, 'test3': 2, 'test4': 3, 'test5': 4}
```

```
words = ['test1', 'test2', 'test3', 'test4', 'test5']
plot_embeddings(M_reduced_plot_test, word2ind_plot_test, words)

print ("-" * 80)
```

```
--------------------------------------------------------------------------------
Outputted Plot:
--------------------------------------------------------------------------------
```



Question 1.5: Co-Occurrence Plot Analysis [written] (3 points)

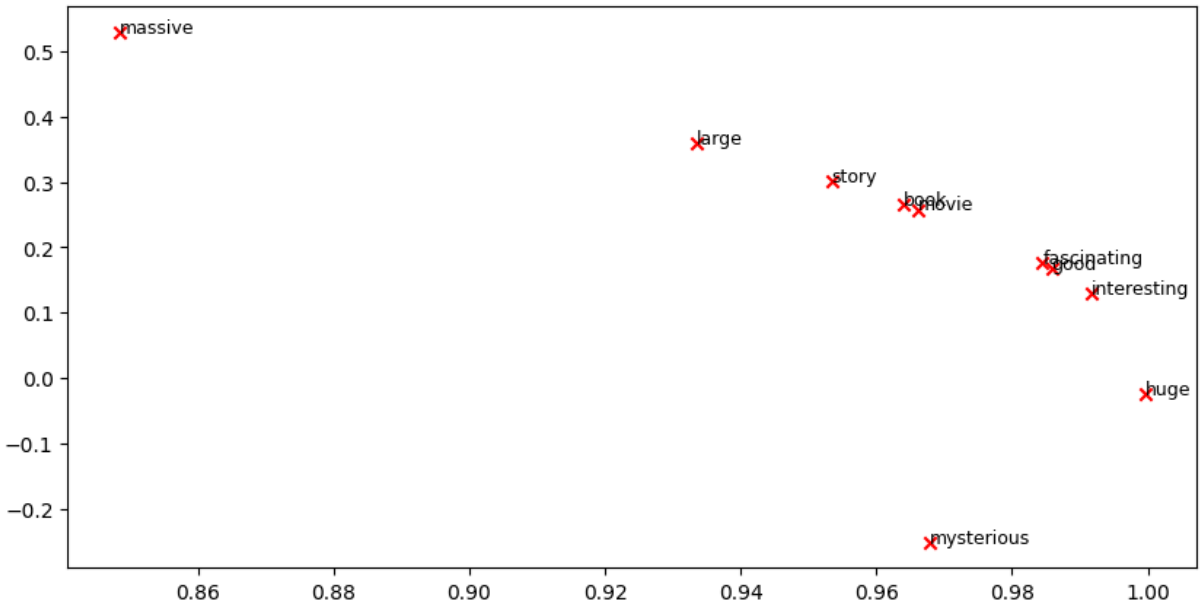
Now we will put together all the parts you have written! We will compute the co-occurrence matrix with fixed window of 4 (the default window size), over the Large Movie Review corpus. Then we will use TruncatedSVD to compute 2-dimensional embeddings of each word. TruncatedSVD returns U*S, so we need to normalize the returned vectors, so that all the vectors will appear around the unit circle (therefore closeness is directional closeness). **Note**: The line of code below that does the normalizing uses the NumPy concept of *broadcasting*. If you don't know about broadcasting, check out Computation on Arrays: Broadcasting by Jake VanderPlas.

Run the below cell to produce the plot. It can take up to a few minutes to run.

In [20]:
```
# -----------------------------
# Run This Cell to Produce Your Plot
# -----------------------------
imdb_corpus = read_corpus()
M_co_occurrence, word2ind_co_occurrence = compute_co_occurrence_matrix(imdb_corpus)
M_reduced_co_occurrence = reduce_to_k_dim(M_co_occurrence, k=2)

# Rescale (normalize) the rows to make them each of unit-length
M_lengths = np.linalg.norm(M_reduced_co_occurrence, axis=1)
M_normalized = M_reduced_co_occurrence / M_lengths[:, np.newaxis] # broadcasting

words = ['movie', 'book', 'mysterious', 'story', 'fascinating', 'good', 'interestin
```

```
plot_embeddings(M_normalized, word2ind_co_occurrence, words)
```

```
Running Truncated SVD over 5880 words...
Done.
```



Verify that your figure matches "question_1.5.png" in the assignment zip. If not, use the figure in "question_1.5.png" to answer the next two questions.

a. Find at least two groups of words that cluster together in 2-dimensional embedding space. Give an explanation for each cluster you observe.

Story, book and movie

Fascinating, good and interesting

b. What doesn't cluster together that you might think should have? Describe at least two examples.

massive, large and huge

Part 2: Prediction-Based Word Vectors (15 points)

As discussed in class, more recently prediction-based word vectors have demonstrated better performance, such as word2vec and GloVe (which also utilizes the benefit of counts). Here, we shall explore the embeddings produced by GloVe. Please revisit the class notes and lecture slides for more details on the word2vec and GloVe algorithms. If you're feeling adventurous, challenge yourself and try reading GloVe's original paper.

Then run the following cells to load the GloVe vectors into memory. **Note**: If this is your first time to run these cells, i.e. download the embedding model, it will take a couple minutes to

run. If you've run these cells before, rerunning them will load the model without redownloading it, which will take about 1 to 2 minutes.

In [26]:
```
def load_embedding_model():
    """ Load GloVe Vectors
        Return:
            wv_from_bin: All 400000 embeddings, each length 200
    """
    import gensim.downloader as api
    wv_from_bin = api.load("glove-wiki-gigaword-200")
    print("Loaded vocab size %i" % len(list(wv_from_bin.index_to_key)))
    return wv_from_bin
wv_from_bin = load_embedding_model()
```

```
[==================================================] 100.0% 252.1/252.1MB downloaded
Loaded vocab size 400000
```

Note: If you are receiving a "reset by peer" error, rerun the cell to restart the download.

Reducing dimensionality of Word Embeddings

Let's directly compare the GloVe embeddings to those of the co-occurrence matrix. In order to avoid running out of memory, we will work with a sample of 40000 GloVe vectors instead. Run the following cells to:

1. Put 40000 Glove vectors into a matrix M
2. Run `reduce_to_k_dim` (your Truncated SVD function) to reduce the vectors from 200-dimensional to 2-dimensional.

In [27]:
```
def get_matrix_of_vectors(wv_from_bin, required_words):
    """ Put the GloVe vectors into a matrix M.
        Param:
            wv_from_bin: KeyedVectors object; the 400000 GloVe vectors loaded from
        Return:
            M: numpy matrix shape (num words, 200) containing the vectors
            word2ind: dictionary mapping each word to its row number in M
    """
    import random
    words = list(wv_from_bin.index_to_key)
    print("Shuffling words ...")
    random.seed(225)
    random.shuffle(words)
    print("Putting %i words into word2ind and matrix M..." % len(words))
    word2ind = {}
    M = []
    curInd = 0
    for w in words:
        try:
            M.append(wv_from_bin.get_vector(w))
            word2ind[w] = curInd
            curInd += 1
```

```
        except KeyError:
            continue
    for w in required_words:
        if w in words:
            continue
        try:
            M.append(wv_from_bin.get_vector(w))
            word2ind[w] = curInd
            curInd += 1
        except KeyError:
            continue
    M = np.stack(M)
    print("Done.")
    return M, word2ind
```

In [28]:
```
# -----------------------------------------------------------------
# Run Cell to Reduce 200-Dimensional Word Embeddings to k Dimensions
# Note: This should be quick to run
# -----------------------------------------------------------------
M, word2ind = get_matrix_of_vectors(wv_from_bin, words)
M_reduced = reduce_to_k_dim(M, k=2)

# Rescale (normalize) the rows to make them each of unit-length
M_lengths = np.linalg.norm(M_reduced, axis=1)
M_reduced_normalized = M_reduced / M_lengths[:, np.newaxis] # broadcasting
```

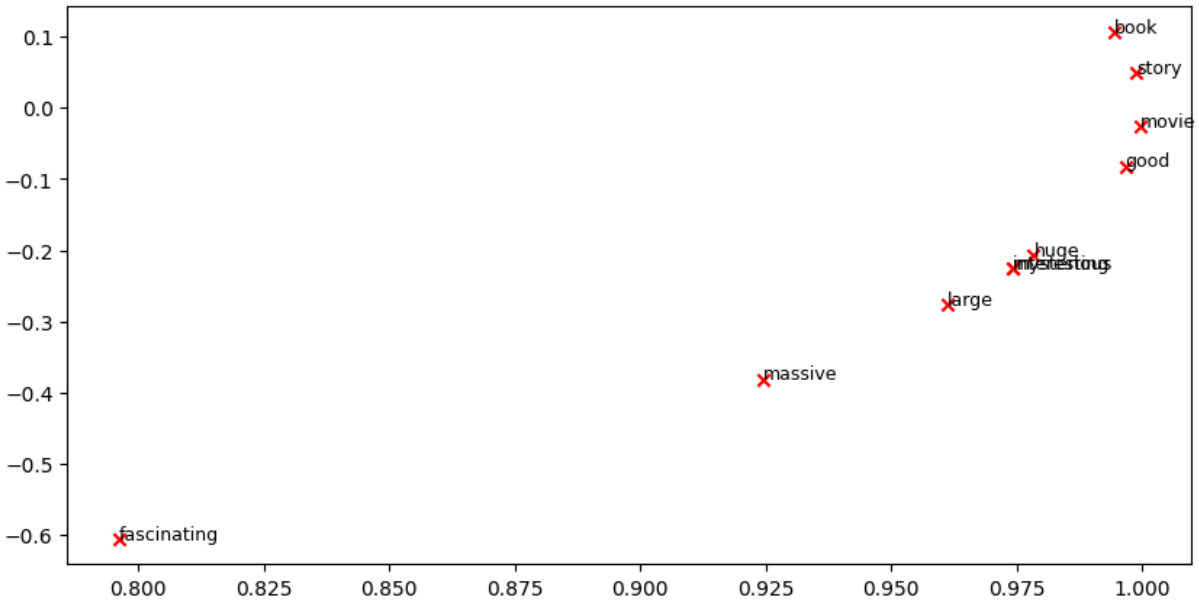
```
Shuffling words ...
Putting 400000 words into word2ind and matrix M...
Done.
Running Truncated SVD over 400000 words...
Done.
```

Note: If you are receiving out of memory issues on your local machine, try closing other applications to free more memory on your device. You may want to try restarting your machine so that you can free up extra memory. Then immediately run the jupyter notebook and see if you can load the word vectors properly. If you still have problems with loading the embeddings onto your local machine after this, please go to office hours or contact course staff.

Question 2.1: GloVe Plot Analysis [written] (3 points)

Run the cell below to plot the 2D GloVe embeddings for `['movie', 'book', 'mysterious', 'story', 'fascinating', 'good', 'interesting', 'large', 'massive', 'huge']`.

In [29]:
```
words = ['movie', 'book', 'mysterious', 'story', 'fascinating', 'good', 'interestin

plot_embeddings(M_reduced_normalized, word2ind, words)
```



Verify that your figure matches "question_2.1.png" in the assignment zip. If not, use the figure in "question_2.1.png" (and the figure in "question_1.5.png", if applicable) to answer the next two questions.

a. What is one way the plot is different from the one generated earlier from the co-occurrence matrix? What is one way it's similar?

Write your answer here.

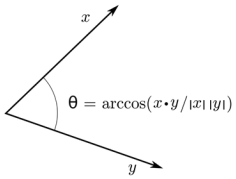
b. Why might the GloVe plot (question_2.1.png) differ from the plot generated earlier from the co-occurrence matrix (question_1.5.png)?

Write your answer here.

Cosine Similarity

Now that we have word vectors, we need a way to quantify the similarity between individual words, according to these vectors. One such metric is cosine-similarity. We will be using this to find words that are "close" and "far" from one another.

We can think of n-dimensional vectors as points in n-dimensional space. If we take this perspective L1 and L2 Distances help quantify the amount of space "we must travel" to get between these two points. Another approach is to examine the angle between two vectors. From trigonometry we know that:



Instead of computing the actual angle, we can leave the similarity in terms of $similarity = cos(\Theta)$. Formally the Cosine Similarity s between two vectors p and q is defined as:

$s = \frac{p \cdot q}{||p|| ||q||}$, where $s \in [-1, 1]$

Question 2.2: Words with Multiple Meanings (1.5 points) [code + written]

Polysemes and homonyms are words that have more than one meaning (see this wiki page to learn more about the difference between polysemes and homonyms). Find a word with *at least two different meanings* such that the top-10 most similar words (according to cosine similarity) contain related words from *both* meanings. For example, "leaves" has both "go_away" and "a_structure_of_a_plant" meaning in the top 10, and "scoop" has both "handed_waffle_cone" and "lowdown". You will probably need to try several polysemous or homonymic words before you find one.

Please state the word you discover and the multiple meanings that occur in the top 10. Why do you think many of the polysemous or homonymic words you tried didn't work (i.e. the top-10 most similar words only contain **one** of the meanings of the words)?

Note: You should use the `wv_from_bin.most_similar(word)` function to get the top 10 most similar words. This function ranks all other words in the vocabulary with respect to their cosine similarity to the given word. For further assistance, please check the **GenSim documentation**.

In [41]:
```
# ------------------
# Write your implementation here.
wv_from_bin.most_similar("bear")

# ------------------
```

Out[41]:
```
[('bears', 0.684942364692688),
 ('grizzly', 0.6034085154533386),
 ('wolf', 0.590702474117279),
 ('stearns', 0.5704878568649292),
 ('lion', 0.5357852578163147),
 ('bearing', 0.5106834173202515),
 ('dog', 0.5077008605003357),
 ('big', 0.49132293462753296),
 ('bore', 0.4897792339324951),
 ('deer', 0.4877018928527832)]
```

Write your answer here.

Question 2.3: Synonyms & Antonyms (2 points) [code + written]

When considering Cosine Similarity, it's often more convenient to think of Cosine Distance, which is simply 1 - Cosine Similarity.

Find three words (w_1, w_2, w_3) where w_1 and w_2 are synonyms and w_1 and w_3 are antonyms, but Cosine Distance $(w_1, w_3) <$ Cosine Distance (w_1, w_2).

As an example, w_1="happy" is closer to w_3="sad" than to w_2="cheerful". Please find a different example that satisfies the above. Once you have found your example, please give a possible explanation for why this counter-intuitive result may have happened.

You should use the the `wv_from_bin.distance(w1, w2)` function here in order to compute the cosine distance between two words. Please see the **GenSim documentation** for further assistance.

In [52]:
```
# ------------------
# Write your implementation here.
w1 = "quiet"
w2 = "soundless"
w3 = "noisy"
print(wv_from_bin.distance(w1, w2))
print(wv_from_bin.distance(w1, w3))
# ------------------
```

```
0.94548017
0.54043067
```

Write your answer here.

Question 2.4: Analogies with Word Vectors [written] (1.5 points)

Word vectors have been shown to *sometimes* exhibit the ability to solve analogies.

As an example, for the analogy "man : grandfather :: woman : x" (read: man is to grandfather as woman is to x), what is x?

In the cell below, we show you how to use word vectors to find x using the `most_similar` function from the **GenSim documentation**. The function finds words that are most similar to the words in the `positive` list and most dissimilar from the words in the `negative` list (while omitting the input words, which are often the most similar; see this paper). The answer to the analogy will have the highest cosine similarity (largest returned numerical value).

In [53]:
```
# Run this cell to answer the analogy -- man : grandfather :: woman : x
pprint.pprint(wv_from_bin.most_similar(positive=['woman', 'grandfather'], negative=
```

```
[('grandmother', 0.7608445286750793),
 ('granddaughter', 0.7200808525085449),
 ('daughter', 0.7168302536010742),
 ('mother', 0.7151536345481873),
 ('niece', 0.7005682587623596),
 ('father', 0.6659888029098511),
 ('aunt', 0.6623409390449524),
 ('grandson', 0.6618767976760864),
 ('grandparents', 0.644661009311676),
 ('wife', 0.6445354223251343)]
```

Let m, g, w, and x denote the word vectors for `man`, `grandfather`, `woman`, and the answer, respectively. Using **only** vectors m, g, w, and the vector arithmetic operators $+$ and $-$ in your answer, what is the expression in which we are maximizing cosine similarity with x?

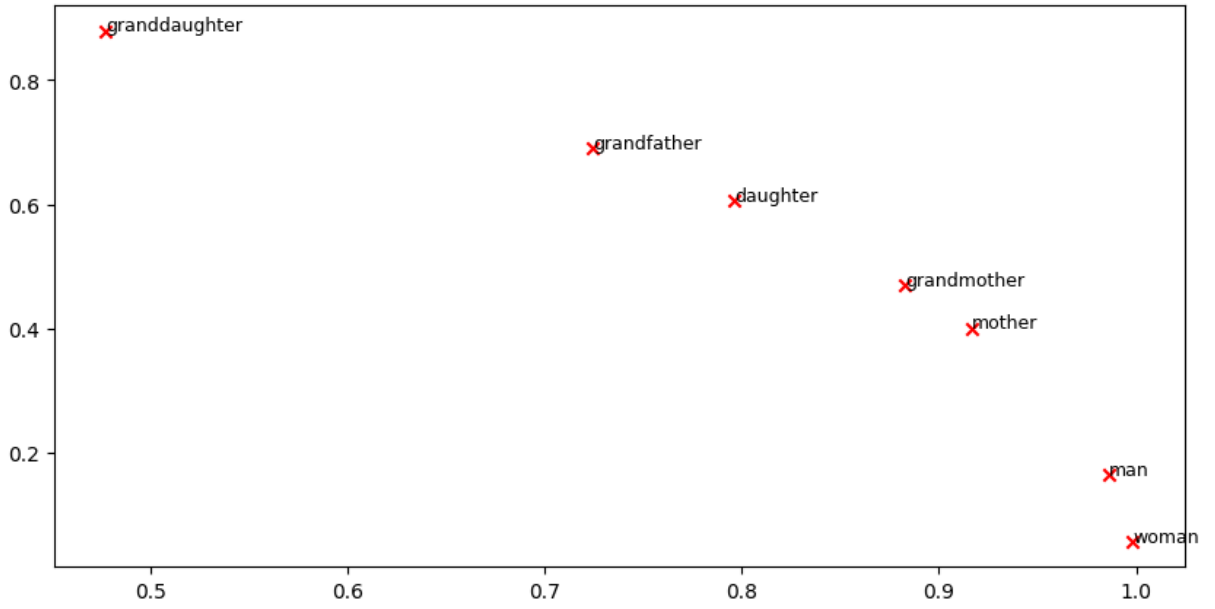
Hint: Recall that word vectors are simply multi-dimensional vectors that represent a word. It might help to draw out a 2D example using arbitrary locations of each vector. Where would `man` and `woman` lie in the coordinate plane relative to `grandfather` and the answer?

w+g-m

Question 2.5: Finding Analogies [code + written] (1.5 points)

a. For the previous example, it's clear that "grandmother" completes the analogy. But give an intuitive explanation as to why the `most_similar` function gives us words like "granddaughter", "daughter", or "mother?

In [54]:
```
plot_embeddings(M_reduced_normalized, word2ind, ['woman', 'man', 'grandfather', 'gr
```



Write your answer here.

b. Find an example of analogy that holds according to these vectors (i.e. the intended word is ranked top). In your solution please state the full analogy in the form x:y :: a:b. If you believe the analogy is complicated, explain why the analogy holds in one or two sentences.

Note: You may have to try many analogies to find one that works!

In [62]:
```
# For example: x, y, a, b = ("", "", "", "")
# ------------------
# Write your implementation here.
x, y, a, b = ("hyperglycemia", "high", "hypoglycemia", "low")

# ------------------

# Test the solution
print(wv_from_bin.most_similar(positive=[a, y], negative=[x])[0][0])
assert wv_from_bin.most_similar(positive=[a, y], negative=[x])[0][0] == b
```

```
low
```

Write your answer here.

Question 2.6: Incorrect Analogy [code + written] (1.5 points)

a. Below, we expect to see the intended analogy "hand : glove :: foot : **sock**", but we see an unexpected result instead. Give a potential reason as to why this particular analogy turned out the way it did?

In [63]:
```
pprint.pprint(wv_from_bin.most_similar(positive=['foot', 'glove'], negative=['hand'
```

```
[('45,000-square', 0.4922032654285431),
 ('15,000-square', 0.4649604558944702),
 ('10,000-square', 0.4544755816459656),
 ('6,000-square', 0.44975775480270386),
 ('3,500-square', 0.444133460521698),
 ('700-square', 0.44257497787475586),
 ('50,000-square', 0.4356396794319153),
 ('3,000-square', 0.43486514687538147),
 ('30,000-square', 0.4330596923828125),
 ('footed', 0.43236875534057617)]
```

Write your answer here.

b. Find another example of analogy that does *not* hold according to these vectors. In your solution, state the intended analogy in the form x:y :: a:b, and state the **incorrect** value of b according to the word vectors (in the previous example, this would be **'45,000-square'**).

In [65]:
```
# For example: x, y, a, b = ("", "", "", "")
# ------------------
# Write your implementation here.
x, y, a, b = ("two", "binary", "ten", "decimal")
```

```
# ------------------
pprint.pprint(wv_from_bin.most_similar(positive=[a, y], negative=[x]))
assert wv_from_bin.most_similar(positive=[a, y], negative=[x])[0][0] != b
```

```
[('ternary', 0.5001227259635925),
 ('decimal', 0.49637529253959656),
 ('numeral', 0.4759579598903656),
 ('duodecimal', 0.46311482787132263),
 ('hexadecimal', 0.4566473662853241),
 ('executables', 0.4558739960193634),
 ('unrooted', 0.45277640223503113),
 ('sefirot', 0.4425036609172821),
 ('corresponds', 0.42566680908203125),
 ('octal', 0.42318835854530334)]
```

Write your answer here.

Question 2.7: Guided Analysis of Bias in Word Vectors [written] (1 point)

It's important to be cognizant of the biases (gender, race, sexual orientation etc.) implicit in our word embeddings. Bias can be dangerous because it can reinforce stereotypes through applications that employ these models.

Run the cell below, to examine (a) which terms are most similar to "man" and "profession" and most dissimilar to "woman" and (b) which terms are most similar to "woman" and "profession" and most dissimilar to "man". Point out the difference between the list of female-associated words and the list of male-associated words, and explain how it is reflecting gender bias.

In [66]:
```
# Run this cell
# Here `positive` indicates the list of words to be similar to and `negative` indic
# most dissimilar from.

pprint.pprint(wv_from_bin.most_similar(positive=['man', 'profession'], negative=['w
print()
pprint.pprint(wv_from_bin.most_similar(positive=['woman', 'profession'], negative=[
```

```
[('reputation', 0.5250176787376404),
 ('professions', 0.5178037881851196),
 ('skill', 0.49046966433525085),
 ('skills', 0.49005505442619324),
 ('ethic', 0.4897659420967102),
 ('business', 0.487585186958313),
 ('respected', 0.485920250415802),
 ('practice', 0.482104629278183),
 ('regarded', 0.4778572916984558),
 ('life', 0.4760662019252777)]

[('professions', 0.5957457423210144),
 ('practitioner', 0.49884122610092163),
 ('teaching', 0.48292139172554016),
 ('nursing', 0.48211804032325745),
 ('vocation', 0.4788965880870819),
 ('teacher', 0.47160351276397705),
 ('practicing', 0.46937814354896545),
 ('educator', 0.46524327993392944),
 ('physicians', 0.4628995358943939),
 ('professionals', 0.4601394236087799)]
```

Write your answer here.

Question 2.8: Independent Analysis of Bias in Word Vectors [code + written] (1 point)

Use the `most_similar` function to find another pair of analogies that demonstrates some bias is exhibited by the vectors. Please briefly explain the example of bias that you discover.

In [77]:
```
# ------------------
# Write your implementation here.

pprint.pprint(wv_from_bin.most_similar(positive=['student', 'asian'], negative=['wh
print()
pprint.pprint(wv_from_bin.most_similar(positive=['student', 'white'], negative=['as
# ------------------
```

```
[('students', 0.5117695927619934),
 ('asia', 0.4890408217906952),
 ('graduate', 0.484187513589859),
 ('asean', 0.4615284502506256),
 ('academic', 0.4578512907028198),
 ('university', 0.4556069076061249),
 ('universities', 0.45448237657546997),
 ('teacher', 0.45402979850769043),
 ('undergraduate', 0.44976696372032166),
 ('singapore', 0.4445837736129761)]

[('black', 0.5223724842071533),
 ('students', 0.5176203846931458),
 ('teacher', 0.5145306587219238),
 ('school', 0.49637413024902344),
 ('brown', 0.4884707033634186),
 ('wearing', 0.48460185527801514),
 ('employee', 0.4693719148635864),
 ('graduate', 0.46549317240715027),
 ('young', 0.4563281238079071),
 ('campus', 0.45620039105415344)]
```

Write your answer here.

Question 2.9: Thinking About Bias [written] (2 points)

a. Give one possible explanation of how bias gets into the word vectors. Your explanation should be focused on word vectors, as opposed to bias in other AI systems (e.g., ChatGPT). You can use specific historical examples to back up your explanations if necessary.

Write your answer here.

b. What is one possible method you can use to mitigate bias exhibited by word vectors? Briefly explain the method and what the goal of the method was.

Write your answer here.

Submission Instructions

1. Click the Save button at the top of the Jupyter Notebook.
2. Select Edit -> Clear Outputs of All Cells. This will clear all the outputs from all cells (but will keep the content of all cells).
3. Select Run -> Run All Cells. This will run all the cells in order, and will take several minutes.
4. Once you've rerun everything, select File -> Save and Export Notebook as -> PDF (If you see errors like "nbconvert failed: Pandoc wasn't found", you can first save it as HTML). Select File -> Save and Export Notebook as -> HTML. This will save the notebook as an

HTML file on your computer. Open the downloaded HTML file in your web browser. In the browser, press Ctrl + P (Windows/Linux) or Cmd + P (Mac) to open the print dialog. In the print dialog, change the destination to Save as PDF and click Save. Make sure all your solutions especially the coding parts are displayed in the pdf, it's okay if the provided codes get cut off because lines are not wrapped in code cells.

5. Look at the PDF file and make sure all your solutions are there, displayed correctly. The PDF is the only thing your graders will see!
6. Submit your PDF on Gradescope.